



***Dissertation on***

**“Hybrid System for Ransomware Detection using ML  
and NLP”**

*Submitted in partial fulfilment of the requirements for the award of degree of*

**Master of Technology  
in  
Computer Science & Engineering**

**UE19CS651 – Project Phase - 2**

***Submitted by:***

**Monica Sneha**

**PES2PG19CS003**

***Under the guidance of***

**Dr. Arti Arya**

**Prof. CSE**

**PES University**

**January - May 2021**

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

**FACULTY OF ENGINEERING**

**PES UNIVERSITY**

**(Established under Karnataka Act No. 16 of 2013)**

**Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India**



## PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)  
Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

### FACULTY OF ENGINEERING

## CERTIFICATE

*This is to certify that the dissertation entitled*

**‘Hybrid System for Ransomware Detection using ML and NLP’**

*is a bonafide work carried out by*

**Monica Sneha**

**PES2PG19CS003**

in partial fulfilment for the completion of Fourth Semester Project Phase - 2 (UE19CS651) in the Program of Study - Master of Technology in Computer Science and Engineering under rules and regulations of PES University, Bengaluru during the period Jan. 2021 – May. 2021. It is certified that all corrections / suggestions indicated for internal assessment have been incorporated in the report. The dissertation has been approved as it satisfies the 4<sup>th</sup> semester academic requirements in respect of project work.

Signature  
Dr. Arti Arya  
Prof. CSE

Signature  
Dr. Sandesh B J  
Chairperson

Signature  
Dr. B K Keshavan  
Dean of Faculty

### External Viva

**Name of the Examiners**

**Signature with Date**

1. \_\_\_\_\_

\_\_\_\_\_

2. \_\_\_\_\_

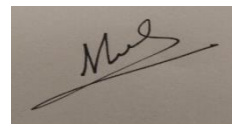
\_\_\_\_\_

## DECLARATION

We hereby declare that the Project Phase - 2 entitled “**Hybrid System for Ransomware Detection using ML and NLP**” has been carried out by us under the guidance of Dr. Arti Arya, Dept of CSE and submitted in partial fulfilment of the course requirements for the award of degree of **Master of Technology in Computer Science and Engineering** of **PES University, Bengaluru** during the academic semester January – May 2021. The matter embodied in this report has not been submitted to any other university or institution for the award of any degree.

**PES2PG19CS003**

**Monica Sneha**



## **ACKNOWLEDGEMENT**

I would like to express my gratitude to Dr. Arti Arya, Department of Computer Science and Engineering, PES University, for her continuous guidance, assistance, and encouragement throughout the development of this UE19CS601 - Project Phase – 2.

I am grateful to the project coordinators, Dr. Saravathi V, for organizing, managing, and helping with the entire process.

I take this opportunity to thank Dr. Sandesh B J, Chairperson, Department of Computer Science and Engineering, PES University, for all the knowledge and support I have received from the department. I would like to thank Dr. B.K. Keshavan, Dean of Faculty, PES University for his help.

I am deeply grateful to Dr. M. R. Doreswamy, Chancellor, PES University, Prof. Jawahar Doreswamy, Pro Chancellor – PES University, Dr. Suryaprasad J, Vice-Chancellor, PES University for providing to me various opportunities and enlightenment every step of the way. Finally, this project could not have been completed without the continual support and encouragement I have received from my family and friends.

## **ABSTRACT**

Ransomware is a malware that gains control of devices such as computers and phones, and demand ransom to return access to the users. In the past years, the growing digital world has been prey to this attack and its rate is expected to grow exponentially in the coming years. In the past two years, there has been an increase in ransomware costs globally. Ransomware negatively impacts the organizations and causes damages in the long run. While the victims recover from such attacks, recovering from the aftereffects is a challenge and time-consuming. The proposed hybrid system intends to detect ransomware using Machine Learning and Natural Language processing techniques. The features extracted of Static and dynamic analysis of ransomware are used. The hybrid system retains the benefits of both the static and dynamic approaches while reducing the limitations.

# TABLE OF CONTENTS

| Chapter No. | Title                                           | Page No.  |
|-------------|-------------------------------------------------|-----------|
| <b>1.</b>   | <b>INTRODUCTION</b>                             | <b>01</b> |
|             | 1.1 Background                                  |           |
|             | 1.2 Problem Statement                           |           |
|             | 1.3 Ransomware                                  |           |
|             | 1.4 Proposed Solution                           |           |
| <b>2.</b>   | <b>LITERATURE SURVEY</b>                        | <b>11</b> |
|             | 2.1 Machine Learning Approach                   |           |
|             | 2.2 Deep Learning Approach                      |           |
|             | 2.3 Natural Language Processing Approach        |           |
|             | 2.4 Summary                                     |           |
| <b>3.</b>   | <b>SYSTEM REQUIREMENT SPECIFICATION</b>         | <b>18</b> |
|             | 3.1 Introduction                                |           |
|             | 3.1.1 Project Scope                             |           |
|             | 3.2 Product Perspective                         |           |
|             | 3.2.1 Product Features                          |           |
|             | 3.2.2 Assumptions, Dependencies and Constraints |           |
|             | 3.2.3 Operating Environment                     |           |
|             | 3.3 Functional Requirements                     |           |
|             | 3.4 Non-Functional Requirements                 |           |
|             | 3.5 System Environment                          |           |
|             | 3.5.1 Hardware requirements                     |           |
|             | 3.5.2 Software requirements                     |           |

|           |                                              |           |
|-----------|----------------------------------------------|-----------|
| <b>4.</b> | <b>PROPOSED METHODOLOGY</b>                  | <b>24</b> |
|           | 4.1 System Architecture                      |           |
|           | 4.2 System Design                            |           |
|           | 4.2.1 The Pre-Processing Subsystem           |           |
|           | 4.2.2 The Feature Generation Subsystem       |           |
|           | 4.2.3 The Model Fitting Evaluation Subsystem |           |
|           | 4.2.4 Summary                                |           |
| <b>5.</b> | <b>IMPLEMENTATION DETAILS</b>                | <b>29</b> |
|           | 5.1 Software Tools                           |           |
|           | 5.2 Modules                                  |           |
| <b>6.</b> | <b>RESULTS AND DISCUSSION</b>                | <b>32</b> |
|           | 6.1 Datasets                                 |           |
|           | 6.2 Pre-Processing Results                   |           |
|           | 6.3 Feature Extraction Results               |           |
|           | 6.4 Random Forest Results                    |           |
|           | 6.5 Random Forest with boosting Results      |           |
|           | 6.6 Neural network Results                   |           |
|           | 6.7 RNN Results                              |           |
|           | 6.8 RNN with attention mechanism results     |           |
|           | 6.9 Discussion                               |           |
| <b>7.</b> | <b>CONCLUSION AND FUTURE WORK</b>            | <b>44</b> |
|           | <b>BIBLIOGRAPHY</b>                          | <b>45</b> |

## LIST OF TABLES

| <b>Table No.</b> | <b>Title</b>                                                      | <b>Page No.</b> |
|------------------|-------------------------------------------------------------------|-----------------|
| 2.4              | Summary of the different approaches used for ransomware detection | 17              |
| 3.1              | Hardware Requirements                                             | 22              |
| 3.2              | Software Requirements                                             | 23              |
| 5.1              | Software Tools                                                    | 29              |
| 6.1              | Table of Classification Metrics                                   | 42              |

## LIST OF FIGURES

| Figure No. | Title                                                                        | Page No. |
|------------|------------------------------------------------------------------------------|----------|
| 1.1        | Severity of Ransomware Attacks ranked by CyberEdge                           | 02       |
| 1.2        | Percentage of victimized organizations and those that pay the ransom         | 03       |
| 1.3        | Technology Preference for security products, CyberThreat Défense Report 2020 | 04       |
| 1.4        | Message by the extortionists to the victim organization                      | 06       |
| 4.1        | Ransomware Detection System Architecture                                     | 25       |
| 4.2        | Pre-processing Subsystem Design                                              | 26       |
| 4.3        | Feature Engineering Subsystem Design                                         | 27       |
| 4.4        | Model fitting and Evaluation Subsystem Design                                | 28       |
| 5.2        | Model Evaluation Flowchart                                                   | 31       |
| 6.1        | API sequence Dataset text file screenshot                                    | 32       |
| 6.2        | Percentage Distribution of the Dataset                                       | 33       |
| 6.3        | Dataset Class Frequency Distribution                                         | 33       |
| 6.4        | N-gram model Implementation result                                           | 34       |
| 6.5        | The IDF computed in TF-IDF model                                             | 35       |
| 6.6        | Confusion Matrix for Random Forest model                                     | 36       |
| 6.7        | Confusion Matrix for Random Forest model with Boosting                       | 37       |
| 6.8        | Confusion Matrix for MLP                                                     | 38       |
| 6.9        | Confusion Matrix for RNN                                                     | 39       |
| 6.10       | Confusion Matrix for RNN with attention                                      | 40       |



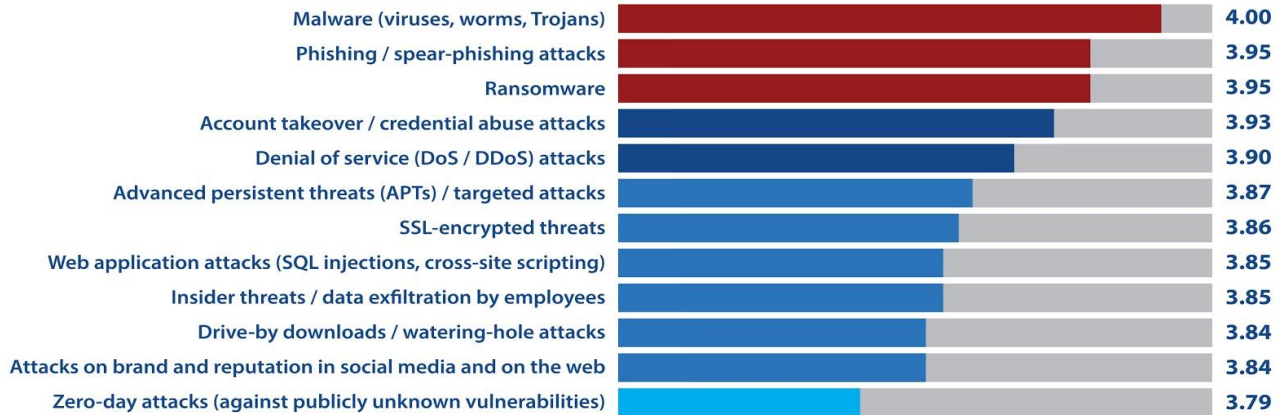
# CHAPTER-1

## INTRODUCTION

Ransomware is a Modern-day Extortion that has been growing exponentially over the years [1]. Ransomware is used by the extortionists to gain admittance to the victim machine and locks out the legitimate user from the system. Ransomware also encrypts the files and prevent access to important documents. Extortionists demand ransom to revert control of the machine, while in many cases the users do not gain complete access post-payment also. In the case of the Organizations, Ransomware blocks access to the business resources and lead to downtime, which in the case of e-commerce businesses can be mortifying. The financial damage of the Ransomware includes the ransom paid and also the restoration costs post the attack. Ransomware is not just an attack on the organizational finances, but also on its reputation, making them liable to their clients and customers. Such attacks have indirect impacts such as lowering of company's stock prices and can mould the future of the company.

Ransomware attacks can be traced back to 1989, but this malware has evolved and remains a relevant problem even today. According to the Cyber Threat Defence report 2020, (as depicted by Figure 1.0) by the CyberEdge group [2], the ransomware attacks is the third most serious cyber threat in the world. One of the main reasons for this attack to persist is the use of Crypto Currencies. The Extortionists demand ransom in the form of Cryptocurrencies such as DASH, Bitcoin, Litecoin, which are not easy to trace. With the growing popularity of Bitcoins, such transactions have become convenient and, extortionists also provide support to the victims for obtaining Bitcoins.

According to Mindsight, there has been a 97% increase in global ransomware costs [3]. and the estimated cost of ransomware in 2021 is 20 Billion Dollars. In India, ransomware has caused extensive financial loses and the pandemic also led to an increase in the number of such attacks.

**Relative Concern for Cyberthreats, by Type (Scale of 1 to 5, with 5 Highest)**


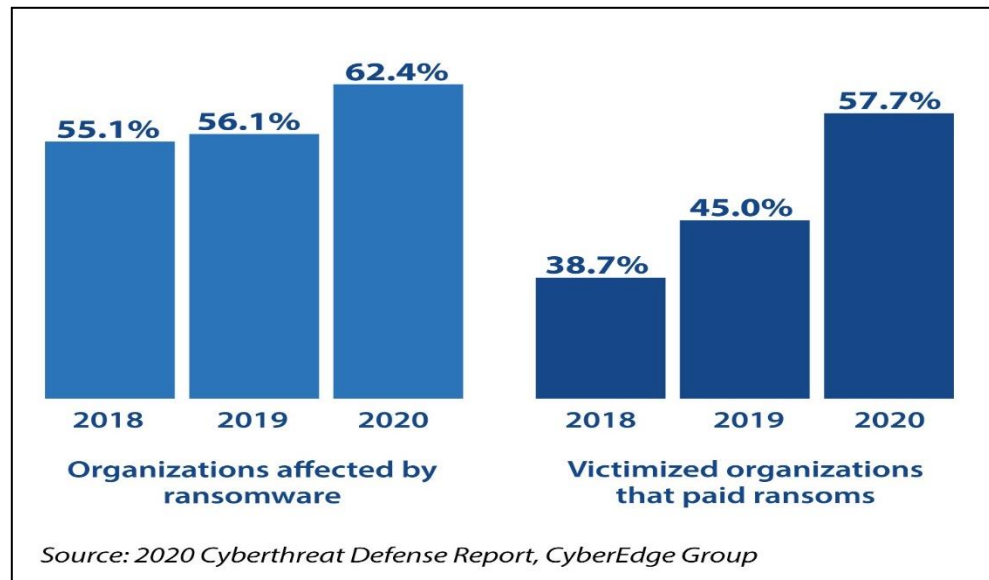
Source: 2020 Cyberthreat Defense Report, CyberEdge Group

**Figure 1.1: Severity of Ransomware Attacks ranked by CyberEdge**

According to Microsoft, India has the 3rd highest targeted encounter rate with Ransomware in Asia and has the 2nd Highest encounter rate in Asia concerning Crypto-currency [4]. This is implied due to the Lower levels of cyber awareness and cyber hygiene practised. During the pandemic credential theft and lateral movement was commonly witnessed in the deployment of Ransomware. Microsoft also believes that better threat intelligence comes from diverse datasets.

Restoration after a ransomware attack is expensive and time-consuming. The aftermath of an attack can cost an organization its reputation and customers. The Norsk Hydro attack is a classic example where 71 million dollars was the monetary damage caused by the ransomware attack and no money was paid as ransom and this enormous amount is used only to recover and restore the business [5]. Unlike Norsk, 57.7% of the victimized organizations tend to pay the ransom in order to retain their locked resources, as estimated and depicted by Figure 1.1. Hence it is plausible to detect ransomware before the full-fledged attack or at least during the initial stages of the attack. Software that can detect Ransomware attacks during the initial stages of the attack and can alert the user or take measures

necessary. These software act as the first layer of defence during an attack. This software consists of Machine Learning models that are trained using the previously observed Ransomware samples.

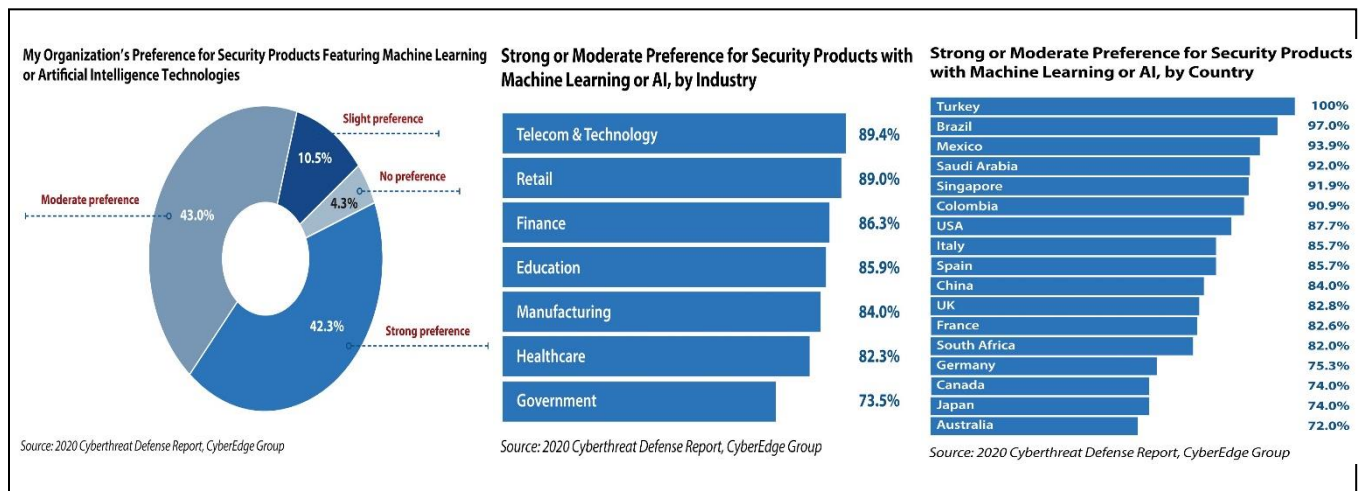


**Figure 1.2: Percentage of victimized organizations and those that pay the ransom**

## 1.1 Background

With the advent of Artificial Intelligence, cybersecurity has found an ally in defending against threats. Cyber threats pose intricate challenges to the experts in facing tailored attacks manually. Cybersecurity is reactive in nature and identifying proactive solutions to attacks is unlikely as predicting new types of attacks is not plausible. Manual threat hunting is an exhausting task in terms of cost and time. Experts assert by Artificial Intelligence to find more equitable solutions to the problems posed by cybersecurity. AI combined with traditional threat hunting techniques has previously rendered the promised detection rates. Artificial intelligence is also used to analyse anomalous behaviour and identify vulnerabilities that can lead to large scale attacks in advance. AI has also contributed to Network security by helping understand network traffic patterns in large networks, and thus in developing better network policies. The CyberThreat Défense report 2020 also

claims that Artificial Intelligence and Machine Learning is preferred by the Organizations, its employees and overall, the industry as depicted by Figure 1.1. While AI is also used by the attackers to improve their attacks, AI has also made the work of the cyber experts easier and less manual.



**Figure 1.3: Technology Preference for security products, CyberThreat Défense Report 2020**

Machine Learning and Natural Language processing are subfields of Artificial Intelligence, that are also extensively used in Network security. While Machine Learning is popular for its uses in cybersecurity for detecting malicious behaviour, tracking down zero-day attacks and automating security tasks, the NLP also has been applied in cybersecurity to solve various problems. NLP has been used to solve one of the classic puzzles of cybersecurity, the phishing attacks. NLP is also used in vulnerability research and detecting persistent threats in DNS. Machine Learning and NLP also prove efficient in resolving another intricacy of cybersecurity that is malware detection. Machine learning algorithms can be trained using observed malware samples and trained models can be used to detect and prevent imminent attacks. Ransomware analysis helps to understand the operation of the ransomware, that can be in return used to detect them. There are two types of analysis techniques, namely static and dynamic analysis and both have their advantages and disadvantages.

Static analysis is the analysis of the malware code that is output by the compilation phase, without executing the malware. Here the source code is examined to identify features, that are common amongst ransomware, but different from the non-malicious software. The advantage of this method is that we can study the ransomware without subjecting a system or an environment to its damages. But this technique can fall prey to code obfuscation. Dynamic analysis involves executing the ransomware in a sandbox or a safe environment to study its behaviour. The characteristics of the ransomware during its run-time are captured and analysed. The advantage of this technique is that it can overcome code obfuscation. Here the challenge lies in setting up an environment for malware execution which can be expensive and time-consuming.

## 1.2 Problem Statement

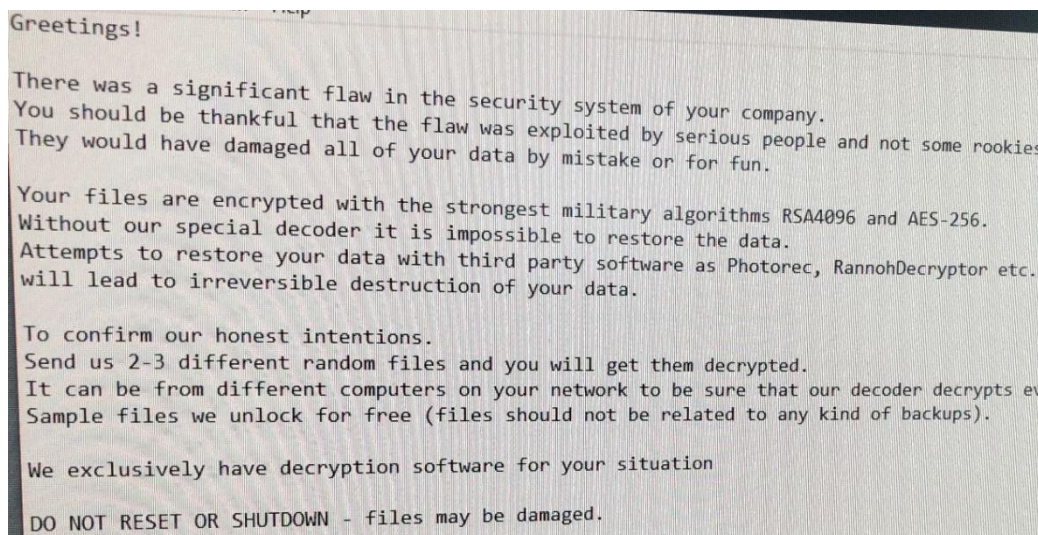
The Ransomware detection process involves several elements to be considered apart from just detecting one. The detection rate solely is not a sufficient observation to evaluate a technique, but the sample size, false-positive rate, the CPU requirements are all important factors to be considered. False positives can be defined as something that is identified as a threat when it is not one. Though these seem to be just a wrong notification, they are misleading and can have a massive impact on the accuracy of the detection tool. False-positive rates suggest the average number of false alarms a detection technique could trigger. A high false-positive rate can drastically reduce the effectiveness of the detection system. It can trigger an incident response for a false breach, that can last for a long time. It can cause a sense of irritation and in the worst case, a sense of ignorance among the security breach responders for future notifications. Hence it is necessary to understand the false-positive rates of the detection system.

Another important factor is considering the CPU requirements because efficiency is as important as effectiveness. It is necessary to estimate the impact of the algorithm on the responsiveness of the system. The time taken by the detection algorithms is another critical determinant of the number of files lost by the time of detection. The sample size helps us understand the scale of the research.

Sample size and samples of ransomware families that are active in the recent years are important considerations for analysis. It is required to have a sample size relatively higher than the number of candidate features.

### 1.3 Ransomwares

The Ransomware attacks are a challenge to detect due to their evolving nature. Ransomware as a service (RAAS) is a platform where skilled developers offer their Ransomware to their affiliates, who buy and use them. RAAS has enabled minimally skilled hackers to acquire and use the services with ease. Customer support and the User interfaces provided to the associates have made Ransomware command and control easier. The anonymous communication mechanisms and Bitcoin payments encourage extortionists to carry out such attacks. Ransomware is also a psychological play, where the extortionists try to trick the user to pay the ransom. Organizations hold a strong sense of responsibility to their customer data and to prevent it from being published can choose to pay off immediately. Attackers use a victim's emotions and his sense of responsibility to trigger him to engage in the deal. Figure 1.3 captures a message left behind by the extortionist for the victim in order to instil fear of the consequences.



**Figure 1.4 : Message by the extortionists to the victim organization**

### 1.3.1 Ransomware Infection Sources

Ransomware attacks the victim machine over various channels. The most common techniques used by the extortionists are spam emails, compromised webpages and also by exploiting the existing vulnerabilities and loopholes in a system. Spam: e-mails with a link or an attached executable file with information that can entice the user to open the email and click on the link. These emails can contain malicious micro attachments such as Trojans that can be a source of Ransomware. Attackers can infect machines via corrupted webpages which can serve as a means for downloading the ransomware on the victim system. When users access on legitimate webpages with files replaced by the hacker, they can download and run the ransomware. Attackers also try to exploit the existing vulnerabilities in the digital infrastructure of the organization and gain access to their resources. These can make their way through rifts in their OS or browsers to the target host in the internal network. Other sources of infection include removable drives, phishing, malicious advertisements, SMS, etc. This is collectively referred to as Social Engineering. The attackers try to use any potential means that can allure the user to do what the attacker or criminal intends.

### 1.3.2 Ransomware Types

Ransomware is basically classified into two types of ransomware, namely Locky Ransomware and Crypto-Ransomware. The former locks the user from accessing the device, whereas the latter encrypts the user files and demands a ransom. The Locky ransomware works by shutting the user out of his device. It infects the user device, elevates its privileges and restricts the user login. Here a non-encrypting algorithm is used, and the files are not encrypted but rendered inaccessible. A pop-up is displayed by the attacker that cannot be closed, preventing the user from accessing the system. Since encryption is not employed the system can be rebooted to safe mode. In contrast, the Crypto-Ransomware encrypts the files on a device and demands a ransom to share the decryption key for restoration. They may encrypt all the files on the system or a few important ones. To save time the

ransomware might also encrypt only a part of the file rendering it useless. The attackers claim safe decryption of all the files post-payment, there is no guarantee that data will not be lost.

### 1.3.3 Ransomware Lifecycle

The Ransomware Lifecycle consists of 6 stages, which regulate its behaviour. The ransomware lifecycle revolves around the three entities, namely the developer, the affiliate, who acquires it and uses it to extort and the target individual or organization. The ransomware profits both the developer and the affiliate, while it detracts the target. The behavioural features of the ransomware are as follows:

1. **Development:** The owner of the ransomware codes the initial program, which is distributed to the affiliates, who use it to wage an attack on the target. The ransomware is sometimes tailored to attack a particular target and based on their experiences; the developer enhances the ransomware. Hence the affiliate keeps the attack going, while the developer keeps updating the ransomware, to prevent its detection.
2. **Infection:** The ransomware infects the target through various sources discussed previously. When the user has successfully lured into downloading the ransomware, it attempts to get higher privilege access. Ransomware can automatically spread itself across the LAN and infect other hosts in the network.
3. **Command and control contact:** After the ransomware have gained access into the host system or server, the ransomware attempts to contact the command-and-control centre(C&C). Ransomware tries to notify the status and also to acquire further commands to intensify the attack. The ransomware may communicate with the centre to also obtain the encryption keys. The ransomware may store the encrypted static IP address or DNS names to contact the C&C centres. Ransomware also can dynamically generate the IP address or the DNS names, making it difficult to detect them. The execution of this stage in the lifecycle is specific to the Ransomware family or type.

4. Encryption key management: There are different ways in which ransomware generate the encryption and decryption key. If the ransomware uses a symmetric algorithm like AES, the decryption key is the same as the encryption key, whereas, in case of the asymmetric encryption, the decryption key is different from the encryption. Ransomware that uses a public key for encryption stores it in the binary encrypted form and does not communicate with the C&C during the encryption but contacts C&C only for the private key during decryption.
5. Encryption: Once the encryption key is generated, the ransomware searches for valuable files and encrypts them. As encryption of the complete file can be time-consuming, ransomware have evolved to encrypt a part of the file and also tend to modify the file extension. The encryption can be symmetric, asymmetric or hybrid.
6. Extortion: Once the valuable files are encrypted, the ransomware puts forth its ransom demand to the user via HTML pages or by changing their desktop background. It also uses text files for ransom claims placed in the encrypted files folder. The ransom note informs the target that his data are encrypted and puts forth a price to unlock his files. It also mentions the mode of payment and also a deadline for the payment.

The behaviour of the ransomware is essential in the development of detection techniques. Understanding the characteristics of the ransomware can help in designing significantly better detection algorithms.

## 1.4 Proposed Solution

The proposed system uses a hybrid methodology to detect ransomware. The Hybrid approach for ransomware detection would retain the benefits of both the static and dynamic approaches while reducing the limitations. Here the static features obtained without the execution of the ransomware and the dynamic features obtained through the execution of the software in a sandbox are used to train the model. The model trained with the static features serves as the first layer of defence and the second model is used as the second layer in the detection of ransomware.

The models are trained on ransomware samples relevant to the current times. Ransomware samples of Families such as WannaCry, Grand crab and Stop ransomware, which has been most active in the past two years [6] are used to train the model.

Natural Language processing techniques, the N-gram model and TF-IDF model are proposed for feature generation and selection. The project Evaluates Machine Learning models such as Random Forest and Deep Learning Models such as Artificial Neural Network with Attention Based Mechanism and the model with a satisfying accuracy is will be used for detection.

## CHAPTER-2

### Literature Survey

Machine Learning techniques have been extensively used in the detection of Ransomware and have performed fairly well. There are many references available in the literature to support the use of Machine Learning techniques in ransomware detection. A few deep learning techniques have also been explored by the researchers in the exposure of this attack, but there exists a small gap where the developing nature of AI where new algorithms are ever-growing and improve the results over time. Below the literature referred to during the research on this system has been explained. The Literature are classified based on the techniques used by the fellow researchers into Machine Learning, deep learning and NLP based approaches.

#### 1.1 Machine Learning Approach

Alhawi et al. [7] illustrates Netconverse, an evaluation model of machine learning classifiers for the detection of ransomware. The Network traffic conversation data during a ransomware attack is analysed to train to achieve a high accuracy rate. The model evaluates 6 machine learning classifiers namely, Bayes Network, Multilayer perceptron, J48, K-Nearest Neighbours, Random Forest and Logistic model tree. J48 out-performs the other classifiers with a detection rate of 97.1%. The highlight of Netconverse model is the pre-processing of the sample data, which includes sample collection technique, refining the sample to eliminate duplicates and non-relevant attributes. The feature reduction has immensely reduced the processing time. The Model can be rendered real-time by adopting a cloud-based setup.

Scalas et al. [8] offers detection of ransomware using finer grain System API features, namely sys API packages, sys API classes and methods. These features are subjected to machine learning classifiers for classification into malware, ransomware and benign classes. The authors have also created an app called R-Packdroid available on Google play based on a similar approach. The paper

claims only system API based features sufficient to detect malicious software, without combining with other types of features. The application is also easy to be ported and implemented on mobile devices, with a good performance. The limitations of using API based approaches are replacement of system API s with similar user implemented codes and finer modifications to the features can evade detection. In the future, the authors intend to develop a small set of highly discriminant features that will not be easy to manipulate.

Stiborek et al. [9] recommends a malware detection based on its interaction with the system resources and the error messages. The training set consists of instances, where each instance is a set of OS resource names and type. It uses clustering of resource names to reduce the impact of randomization. This approach is limited if error messages are not invoked by the malware. Arabo et al. [10] offers a dynamic analysis technique to obtain the ransomware API features, which is used for evaluation of 9 Machine Learning classifiers and a neural network. This study intends to understand the relationship between a process behaviour and its nature and use it in ransomware detection. Random Forest outperforms other classifiers with a detection rate of 75.01%. 1. The authors claim that the benefit of this solution is that it does not need a signature database, but a dataset of ransomware and non-ransomware data. Improving the dataset can increase the detection rate of the classifiers.

Shaukat and Ribeiro [11] proposes Ransomwall, a layered mechanism for ransomware prevention, detection and backup. It Uses a hybrid approach with honey-trap for early detection, ML for zero-day intrusion detection and file backup to prevent data loss. This hybrid approach overcomes the limitation of both static and dynamic and is capable of real-time setup. Egunjobi et al. [12] uses both static and dynamic analysis features to ransomware detection and uses four machine learning classifiers to evaluate its approach. Random forest and SVM achieve the highest detection rate of 99.5%. The study also aimed to reduce the false positive rate and has gained a False Positive Rate (FPR) of 1%.

Poudyal et al. [13] proposed a ransomware detection framework using machine learning for static analysis of the ransomware characteristics at the Dynamic link library (DLL) and assembly level. Ransomware binaries are reverse engineered to gather codes at different levels. The output of the static analysis is used to train 8 Machine Learning Models. The average accuracy is 92.11%, while Random Forest has achieved the highest accuracy of 97.95%. The authors strive to expand their sample size for future experiments. The paper Lee et al. [14] offers to detect the ransomware infected files transferred to the backup system in real-time. Machine learning is used to classify the infected files by entropy analysis. Logistic regression, K Nearest neighbour, Random Forest, Decision tree, Support Vector Machine (SVM) and Multilayer Perceptron (MLP) are used for classifying the infected files and most of the classifiers have performed well. This research can be further expanded to user-based backup systems and for different file formats.

Khan et al. [15] proposes DNAact-Ran, A Digital DNA Sequencing Engine for Ransomware Detection Using Machine Learning. In this approach, the ransomware dataset is pre-processed to identify DNA sequences for ransomware samples. MOGWO and BCS are used for feature selection and, DNA sequences are generated which serve as the training data for the classifier. The active learning classifier achieves a detection rate of 87.9%. Cusack et al. [16] offer a ransomware detection solution leveraging a programmable forwarding engine (PFEs). PFEs allow the collection of per-packet, network monitoring data at high rates. This data is used to monitor the network traffic between an infected computer and the command and control (C&C) server to extract high-level flow features from this traffic and use this data for ransomware classification using random forest. They have also worked on a Crypto-based Cerber ransomware classifier specifically that, exhibits a 10-fold CV score average comes in at 0.905.

## 2.2 Deep Learning Approach

Vinayakumar et al. [17] evaluates the effectiveness of shallow and deep learning networks for detection and classification of ransomware. The ransomware is executed in the sandbox to acquire

API sequences used for binary and multi-class classification of ransomware families. This work has achieved a perfect detection rate with the use of Multi-layer perceptron. Eight Ransomware families have been used for multi-class classification, which has a good true positive rate for ransomware families except for Crypto wall and Crypto locker families. Zhang et al. [18] propose a static analysis framework with N-gram opcodes to train deep learning models Convolutional Neural Network and Recurrent Neural Network. The classification results in an average detection rate of 87.5% and, for the Petya family, the proposed framework achieves 100% Precision, 100% Recall and 100% F1-score.

Rhode et al. [19] proposes a dynamic detection technique that, exposes a malicious executable within 5 seconds of its execution. This is achieved by the use of a 5 seconds snapshot of the malware behavioural data and Recurrent neural networks. The model is capable of detection within 5s with 94% accuracy and an accuracy of 89% for unseen malware within 1 second of execution. The authors intend to further expand the model using non-windows executables sample, sliding window approach with 5 seconds snapshot of activity to detect malicious software, post-detection blocking action and develop it into a portable anti-malware application.

Homayoun et al. [20] proposes Deep Ransomware Threat Hunting and Intelligence System (DRTHIS) to distinguish ransomware from benign and identify the ransomware families. DRTHIS utilizes Long Short-Term Memory (LSTM) and Convolutional Neural Network (CNN), two deep learning techniques using soft-max algorithm, for classification. The model is capable of classifying unseen samples of CryptoWall, TorrentLocker and Sage families. Agrawal et al. [21] proposes the LAMP model, a dynamic analysis technique to obtain emulation sequences, that serve as input to the LSTM with attention mechanism for ransomware detection.<sup>1</sup> The Authors have observed a pattern of small API sequences being repeated multiple times and have used attention mechanism to capture it.

Jinsoo et al. [22] proposes a two-stage mixed ransomware detection method with Markov chain detection at the first stage and Random Forest classifier at the second stage. This work evaluates Random Forest, Logistic regression, Deep Neural Network and Markov chain for ransomware detection. Random Forest derives the best detection rate of 97.3% hence is used in the second layer of the model. Al-Hawawreh and Sitnikova [23] proposes a Deep Neural Network (DNN) with Batch normalization model for classifying Ransomware. The input to the classifier is the feature vector obtained by hybrid feature engineering using Convolutional Autoencoders (CAE) and Variational Auto encoders (VAE). The model was able to make predictions from the hybrid feature engineered feature set with an accuracy of 92.53%.

## 2.3 Natural Language Processing Approach

Zhang et al. [24] proposes a static analysis approach where n-gram of opcodes are evaluated with Term Frequency-Inverse document frequency (TF-IDF) to obtain the feature vectors which serve as the train and test set for multi-class classification of ransomware families and binary classification of the ransomware and benign samples. Random Forest algorithm has the best results for binary classification with a recall of 99.8%. But the multi-class classifier is not able to precisely discriminate between CryptoWall, Cryrar and Reveton. Hamed et al. [25] proposes a feature selection -based network intrusion detection method using Recursive feature addition (RFA) and Bigrams to encode payload strings and uses SVM for classification. Use of tri-grams and Ensemble classifiers could be explored. Poudyal et al. [26] illustrates a multi-level ransomware detection framework based on NLP and ML techniques. The ransomware behaviour is analysed at three levels DLL, Assembly and function call levels in a big data environment Apache Spark. Since Apache is used, more ransomware binaries could have been considered for the experiment. Code randomization, junk code insertion, polymorphic codes can evade detection and hence should be considered.

Chen et al. [27] offers a tool to extract a sequence of events provoked by ransomware families and identify the most distinctive features using TF-IDF, Fisher's Linear Discriminant Analysis (LDA) and Extremely Randomized Trees (ET). The model successfully captures unique patterns and their behaviours that are missed out by static analysis. The model can be further expanded using cloud-based Machine learning library.

## 2.4 Summary

This section briefly summarizes the prominent techniques from the Literature that has used unique and more reliable means that have inspired this project. While most surveys focus on ransomware detection using Machine learning techniques, the project intends to explore the sub-domains of Artificial Intelligence. The Literature review consists of detection techniques that employ Machine Learning, Deep Learning and Natural Language Processing techniques, which are sub-fields of AI for ransomware detection. Though the reviewed works have good detection rates, a few open concerns are identified and discussed. The primary objective of the review is to highlight concerns of the proposed system that prevent it from becoming effective in real-time systems.

In the below Table 2.4, we have discussed the different approaches we have reviewed in the literature alongside their pros and cons in order to highlight the choice of technologies for this project. Though Machine Learning techniques are used often in Malware detection, one of the algorithms that stand out in most of the researches is Random Forest, which offers a good detection rate for ransomware. This algorithm if implemented along with a good False positive rate can make a beneficial model for detecting ransomware. Another principal yet not so popular technology used in the detection of ransomware is natural language processing-based N-gram and TF-IDF model that generate features that renders a better detection accuracy. The literature also throws light on the importance of the pre-processing and Feature generation in achieving an efficient model with ample accuracy. The below table highlights the pros and cons of some of the approaches used to tackle ransomware.

**Table 2.4 Summary of the different approaches used for Ransomware Detection**

| Approach                   | Techniques                                                                             | Pros/cons                                                                                                                                                                                           |
|----------------------------|----------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Machine Learning           | Random Forest<br>Support Vector Machine<br>K nearest Neighbours<br>Logistic Regression | <ul style="list-style-type: none"><li>• Good accuracy rate on network datasets and static datasets</li><li>• Does not offer good accuracy on the dynamically generated API features</li></ul>       |
| Deep Learning              | Deep Neural Network<br>Convolutional Neural Network<br>Recurrent Neural Network        | <ul style="list-style-type: none"><li>• Offers a good false positive rate alongside the TPR</li><li>• Multiclass classification of ransomware families does not offer significant results</li></ul> |
| ML or DL combined with NLP | N -gram Model<br>TF-IDF model                                                          | <ul style="list-style-type: none"><li>• Helps generate features that enhance the classifier output</li></ul>                                                                                        |

## CHAPTER-3

### PROJECT REQUIREMENTS SPECIFICATION

In this chapter, the requirements that are fulfilled by the project are discussed in detail. The perspective endeavoured by the project, the features it is intended to offer and the scope of the project is discussed in the upcoming sections. The Environment in which the product is executed, the assumptions and dependencies involved along with the testing requirements are discussed below. This chapter outlines the expectations of the project and the elements required to fulfil these expectations.

#### 3.1 Introduction

The proposed system is expected to expose malicious files that can infect and gain control of the host machine. The proposed system uses a three-layered defence system to track down ransomware before they can take over the infected system. Ransomware exploits the vulnerabilities in the system and the naive nature of the users to gain access to the machine and abuse the victim. The principal purpose of the development of this system is to track down the ransomware before they lock the user out or encrypt the files in a system.

##### 3.1.1 Project Scope

In the digital world, where every organization, business and individuals possess digital assets, there is a massive probability for a cyber threat, that tries to access and use the asset to its own interests. The very existence of an asset is the possibility of an attack and with the growing rate of such attacks, a ransomware detection system will gain its spot in every router, machine, PC and phones. This project intends to develop a Ransomware detection model which can be deployed at the endpoints and can scan incoming files for malicious features and activities. The system is a means to detect the ransomware before it can gain command of the system, while also trying to reduce the false positives, as false positives can lead to false alarms and direct efforts in the wrong alerts. The

project also uses ransomware samples that are most active during the current times, to make the hybrid system more relevant to the current times.

## 3.2 Project Perspective

This section explains the approach undertaken to fulfil the objectives of the project. As previously discussed in section 2, several approaches are already been tested for the detection of the ransomware and the perks of the same are compared. After analysing the various approaches available in the literature, which were fruitful yet could be more effective and efficient, we propose a perspective which uses the already available solutions with a humble yet valuable additional strategy, to make detection process more productive and pleasant.

The ransomware problem is approached with a Machine Learning and deep learning approach, which now a commonly used for detection of many cyber threats along with an addition of Natural language processing techniques. The Project intends to evaluate the different Machine Learning and Deep Learning techniques for detection of ransomware to identify a fitting model with ample accuracy and low false positives. The model is supplied with features that are engineered using the N-gram model and the TF-IDF model to enhance their performance. A Deep Learning model with attention mechanism will be used to capture the repetitive lifecycle patterns observed in the ransomware. The additional Feature engineering techniques and the attention mechanism is expected to make the model more resilient and productive.

### 3.2.1 Product Features

The three major features offered by the project is to detect the ransomware at three levels; during its entry into the network, after its download, and before it executes completely and gains control of the system. Other additional features it offers is a low false-positive rate to avoid annoying false alarms that trigger unnecessary actions. The project also uses appropriate samples of ransomware that have been operating currently and whose attacks are more probable, thus making the system more

applicable to the prevailing scenario. The model can be consumed by the end user applications that intend to offer these features and also can be deployed at endpoints for security.

### 3.2.2 Assumptions, Dependencies and Constraints

Numerous complications arise in heterogeneous systems due to a multitude of factors. The constraints set in place in the development of this system are described below:

1. The samples collected are of ransomware that is active in the current period of time and the system will fail to detect new strands of ransomware engineered lately.
2. The training requires a large and diverse set of data and the processing of this large and diverse set needs reasonable processor speed and RAM. The cloud-based solutions such as AWS cater to the needs of developing this model, while the traditional computing may not be very flexible.
3. Since this research involves handling of ransomware samples that are dangerous and cause severe damage to one's system. It is necessary to handle the samples with utmost care to prevent unpleasant accidents.
4. Though the detection of ransomwares based on network features has a reasonable accuracy, in real world scenario where the field of networks is ever changing, the model will require constant updates in order to remain functional.
5. A ransomware attack is an complex threat and though a detection application can serve as an first layer of defence, an trained team of cybersecurity professional with a well established cyber policies is necessary to keep such attacks at bay.

### 3.2.3 Operating Environment

Though the model is intended for a network endpoint such as a router, it is capable of serving its purpose in any system that detects malwares or ransomwares. The model can be reused in systems that intend to capture malwares using network traffic, downloaded files or files in process of

execution. The most suitable network endpoints for deployment would be routers, yet the model can be included in any application that renders similar functions.

### 3.3 Functional Requirements

The basic functionalities expected of the project are listed below.

1. To Detect Ransomwares incoming over the network or via other transmission medium, using the network features, is the function of the first layer of the detection system
2. To Detect Ransomwares that are downloaded but not yet executed, using the static features, is the function of the second layer of the detection system
3. To Detect Ransomwares that have begun execution before they gain control of the system, using the Dynamic features, is the function of the third layer of the detection system
4. To provide live alerts to the admin in case of any ransomware breach and perform the specified action by the admin, if pre-programmed

### 3.4 Non-Functional Requirements

The detection rate solely is not a sufficient observation to evaluate a technique, but the sample size, false-positive rate, the CPU requirements are all important factors to be considered.

1. False Positive rate: False positives can be defined as something that is identified as a threat when it is not one. Though these seem to be just a wrong notification, they are misleading and can have a massive impact on the accuracy of the detection tool. False-positive rates suggest the average number of false alarms a detection technique could trigger. A high false-positive rate can drastically reduce the effectiveness of the detection system. It can trigger an incident response for a false breach, that can last for a long time. It can cause a sense of irritation and in the worst case, a sense of ignorance among the security breach responders for future notifications. Hence it is necessary to understand the false-positive rates of the detection system.

2. **Productivity:** Another important factor is considering the CPU requirements because efficiency is as important as effectiveness. It is necessary to estimate the impact of the algorithm on the responsiveness of the system. The time taken by the detection algorithms is another critical determinant of the number of files lost by the time of detection.
3. **Sample size:** The sample size helps us understand the scale of the research. Comparison of just the detection rates irrespective of the sample size is meaningless. Sample size and samples of ransomware families that are active in the recent years are important considerations for analysis. It is required to have a sample size relatively higher than the number of candidate features.

### 3.5 System Environment

Initially the pre-processing, feature engineering and the model fitting was performed on a Laptop with the below mentioned hardware specification. As the size and dimensions of the dataset grew the model was moved to the AWS cloud setup.

#### 3.5.1 Hardware Requirements

The initial development was done on a laptop with the following specifications:

**Table 3.1: Hardware Requirements**

|                        |                                                      |
|------------------------|------------------------------------------------------|
| <b>Processor</b>       | Intel(R) Core (TM) i5-8250U CPU @ 1.60GHz, 1801 Mhz, |
| <b>Number of CPU's</b> | 4 Core(s), 8 Logical Processor(s)                    |
| <b>RAM</b>             | 8GB                                                  |
| <b>Cache</b>           | 32KB, 256KB, 3MB                                     |

### 3.5.2 Software requirements

The software requirements of the project are tabulated below. During the later stages of development the system was developed on the AWS cloud and the cloud resources thus used are also tabulated in the table 3.5.2.

**Table 3.2: Software Requirements**

|                                |                                                  |
|--------------------------------|--------------------------------------------------|
| <b>AWS cloud environment</b>   | AWS SageMaker                                    |
| <b>Database</b>                | AWS S3                                           |
| <b>Development language</b>    | Python 3.7                                       |
| <b>Deep Learning Framework</b> | Tensorflow,<br>Keras                             |
| <b>Third party Libraries</b>   | Pandas<br>Numpy<br>NLTK<br>Scikit<br>Metaplotlib |

## CHAPTER-4

### PROPOSED METHODOLOGY

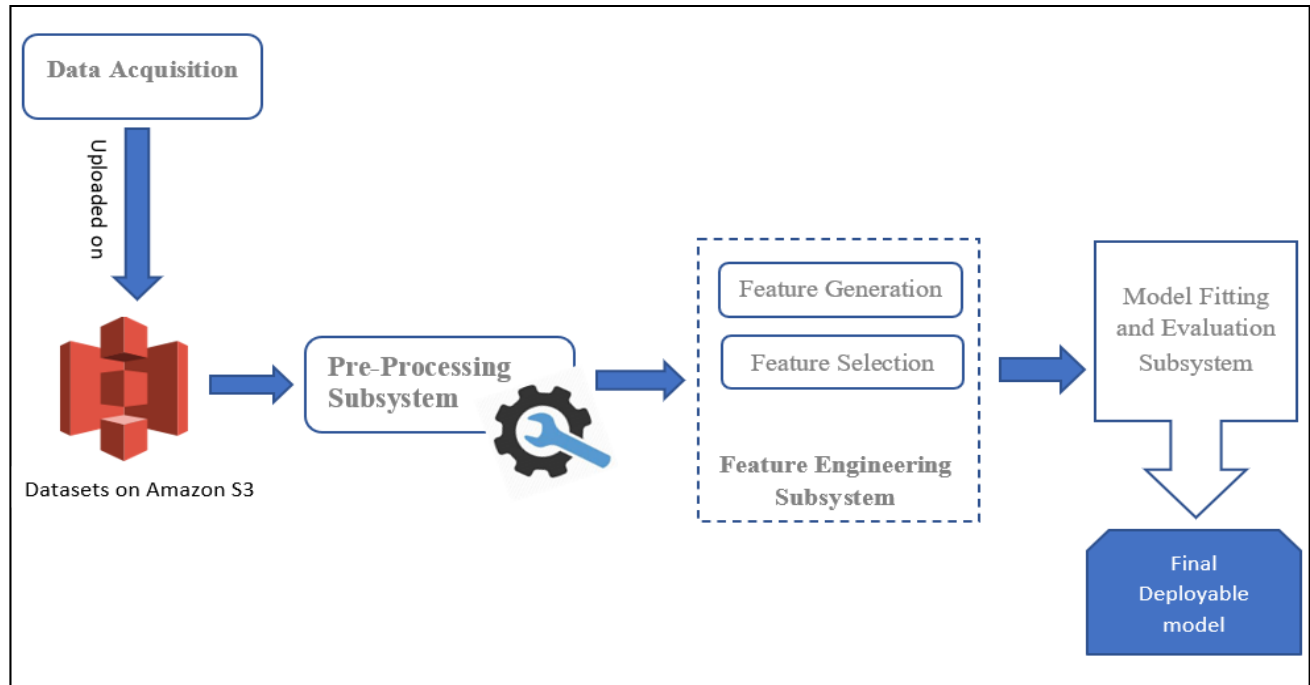
The project proposes a layered approach to Ransomware detection where the first layer of defence is the machine learning model trained using the network traffic dataset of the ransomware. This model is capable of detecting ransomware based on the incoming traffic of the files. The second layer of defence is offered by a model trained on the static analysis feature of the ransomware. This layer scans the files before their execution, so as use the static features of the download to identify if it is ransomware. The third layer of defence is offered by a machine learning model that is trained using the dynamic analysis features of ransomware, which intends to detect ransomware before it completes the execution and gains control of the system.

#### 4.1 System Architecture

The overall system architecture is illustrated in Figure 4.1. The dataset with ransomware and goodware samples acquired from various sources are uploaded to the Amazon S3 data bucket. The detection system basically consists of three subsystems namely, the pre-processing, feature engineering and model fitting-evaluation subsystem. The pre-processing subsystem is intended to clean the raw datasets by eliminating duplicates, reducing the dimension of the datasets and by eliminating irrelevant samples. The feature engineering subsystem performs the two major tasks of feature generation and feature selection. The N-gram model and the TF-IDF model is used for the generation of features and the N-gram and TF-IDF scores decide the selection of the features. The model fitting and evaluation subsystem is responsible for training-testing models and identifying a model with the best accuracy. The system outputs a model capable of ransomware detection.

#### 4.2 System Design

In this section, every subsystem illustrated in the system Architecture Figure 1.4 is explained in detail. These subsystems contribute specific tasks in the implementation of the ransomware detection system.

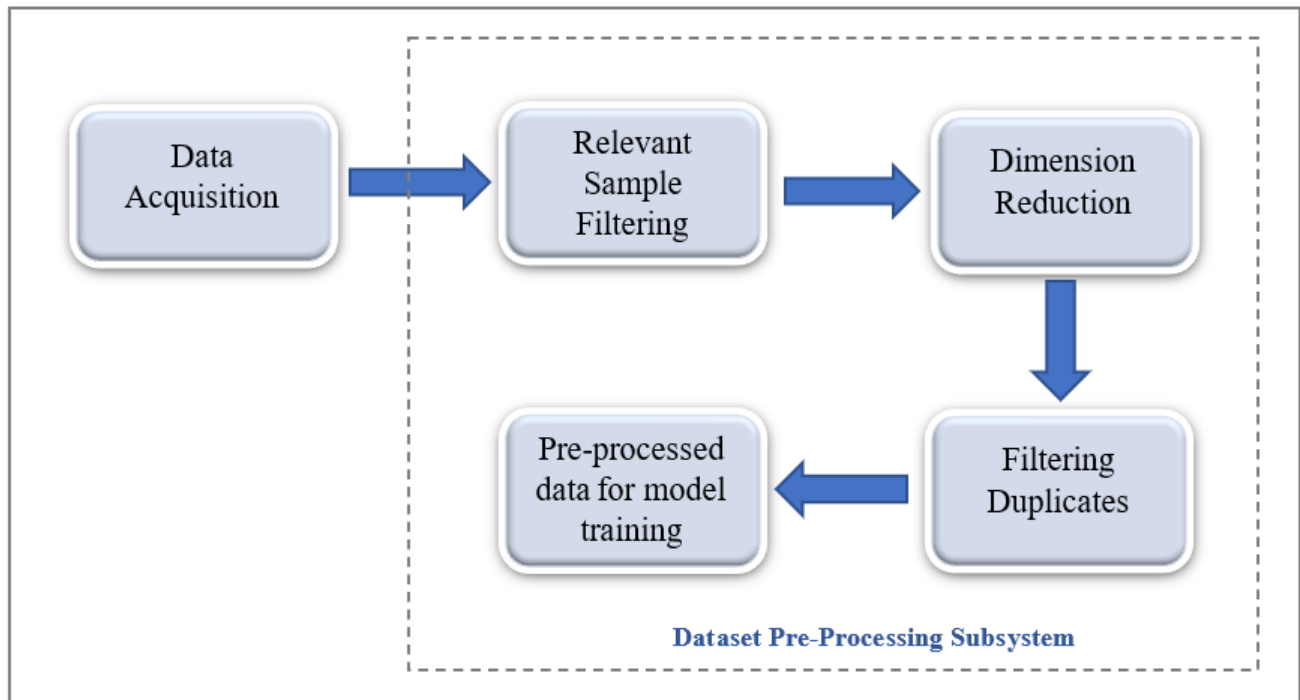


**Figure 4.1: Ransomware Detection System Architecture**

#### 4.2.1 Pre-processing subsystem

The pre-processing subsystem performs an important task of preparing the dataset for the model training. This subsystem consists of four phases namely, relevant sample filtering, dimension reduction and duplicate filtering. Each of these tasks plays an important role in determining the output of the model. The dataset obtained consists of ransomware samples of all times. While most of them are recent and relevant, it also includes a few obsolete samples, hence it is necessary to filter the dataset for relevant samples, to not exhaust the resources and efforts in the futile direction. Since the dataset has been procured from multiple sources, there are many duplicates present in them, which can compromise the integrity of the system. Hence it is necessary to filter the dataset

for duplicates. Dimension reductions is another important aspect of pre-processing that determines the important features of a dataset that require attention, while the dataset consists of many features or columns. Figure 4.2.1 illustrates the pre-processing subsystem.

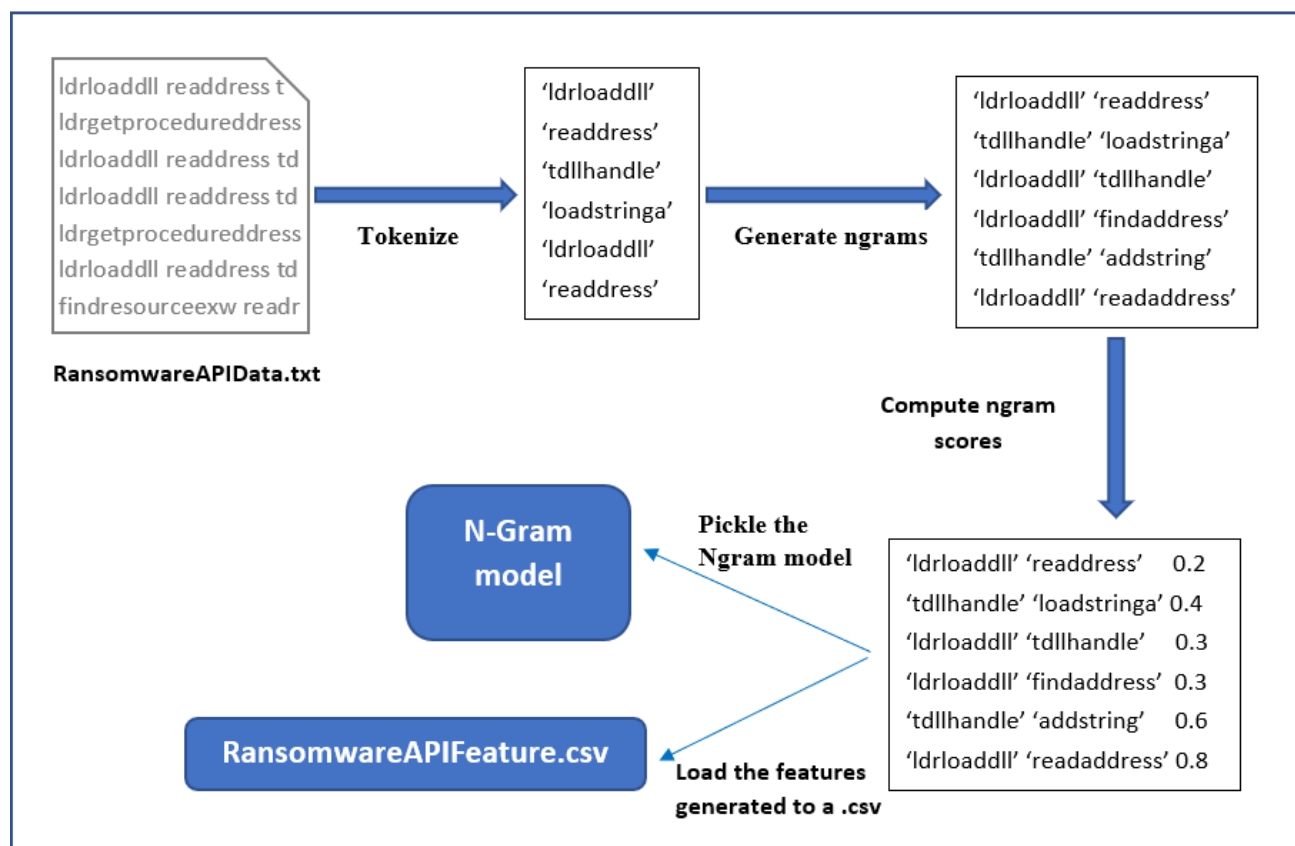


**Figure 4.2: Pre-processing Subsystem Design**

### 4.2.2 Feature Engineering subsystem

The feature engineering subsystem is the central unit of the system architecture. It determines the quality of the model that the system will generate for ransomware detection. The ransomware and goodwill API dataset obtained is a text file where every line in the text file corresponds to the API sequence of either a goodwill or ransomware. The first step in feature engineering is the tokenization of these sequences, to obtain tokens, where each token corresponds to an API. After tokenization, the tokens are used to generate Ngrams, where if  $n=2$ , bigrams are obtained and if  $n=3$ , trigrams are obtained. The next step in the Feature engineering process is the computation of

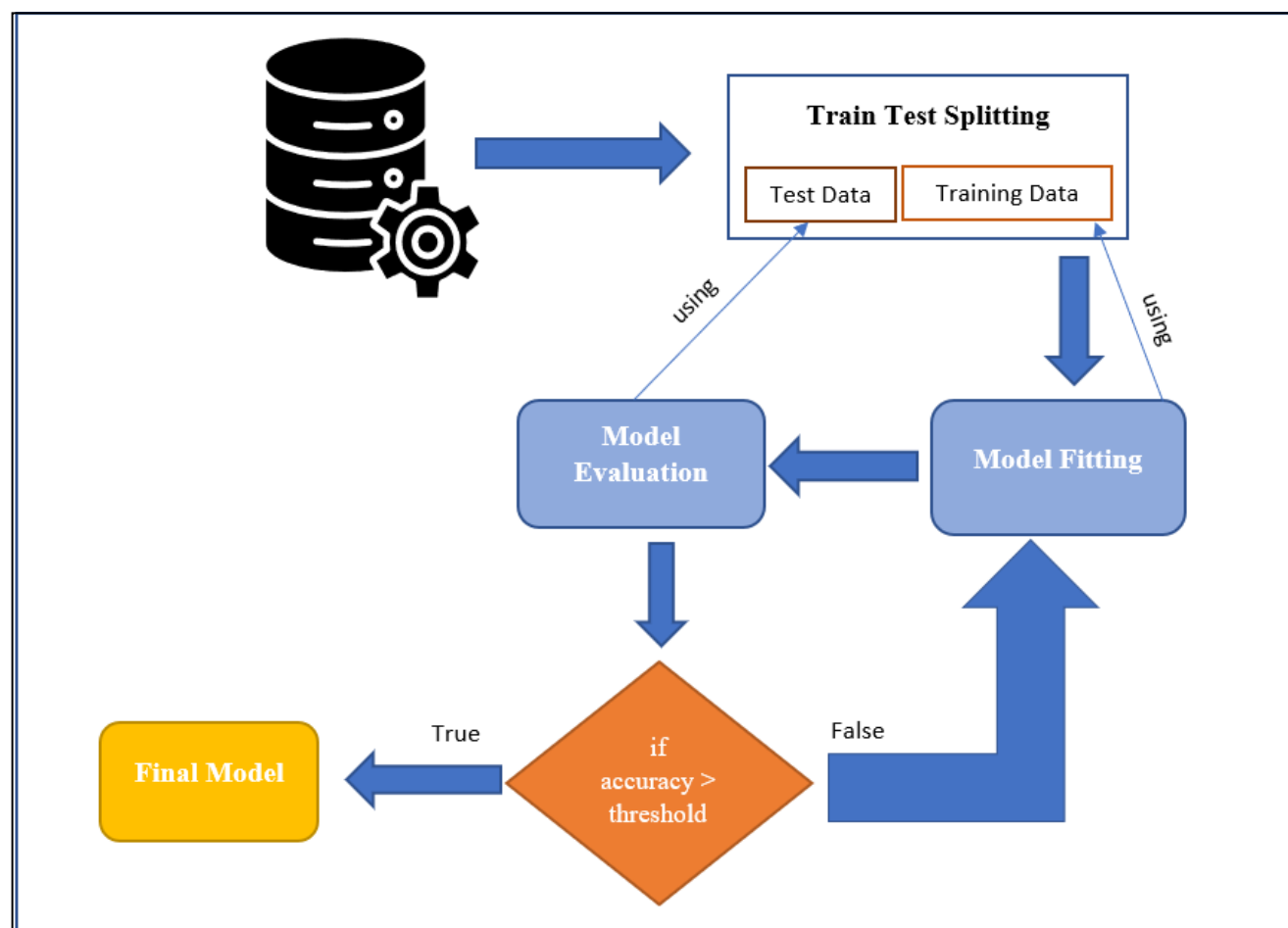
the Ngram scores, which can be defined as the frequency of the occurrence of the two API sequences in the same order. These Ngram scores are unique for sequences of API and help differentiate a ransomware sample from that of a normal file. The Ngram model is pickled and saved for future uses and the features generated are loaded to a .csv file. Figure 4.3 depicts the Feature Engineering subsystem with all its phases.



**Figure 4.3: Feature Engineering Subsystem Design**

## 4.2.3 Model Fitting and Evaluation Subsystem

This subsystem consists of two main phases the Model fitting phase and the model evaluation phase where the model may be accepted or rejected based on its accuracy. The Features generated by the Feature Engineering subsystem and the pre-processed features are split into training and test dataset. The train test split used is 70-30. The training data is then used for model fitting and the test data is used in model evaluation. the generated model is then subjected to the evaluation phase where if the accuracy achieved by the model is greater than the agreed-upon threshold value then the model is accepted else the model is rejected. The final model thus obtained is capable of detecting ransomwares. Figure 4.4 illustrates the process of model fitting and evaluation.



**Figure 4.4: Model Fitting and Evaluation Subsystem Design**

## CHAPTER-5

### IMPLEMENTATION DETAILS AND PSEUDOCODE

This chapter describes the details of the implementation of the Ransomware detection system. The first phase involves the implementation of the Ngram model and the random forest classifier. The Deep learning models with attention mechanism are scheduled for the next phase and are not discussed here. Here the section 5.1 describes the software tools used in the implementation and section 5.2 gives a brief overview of the modules implemented.

#### 5.1 Software tools

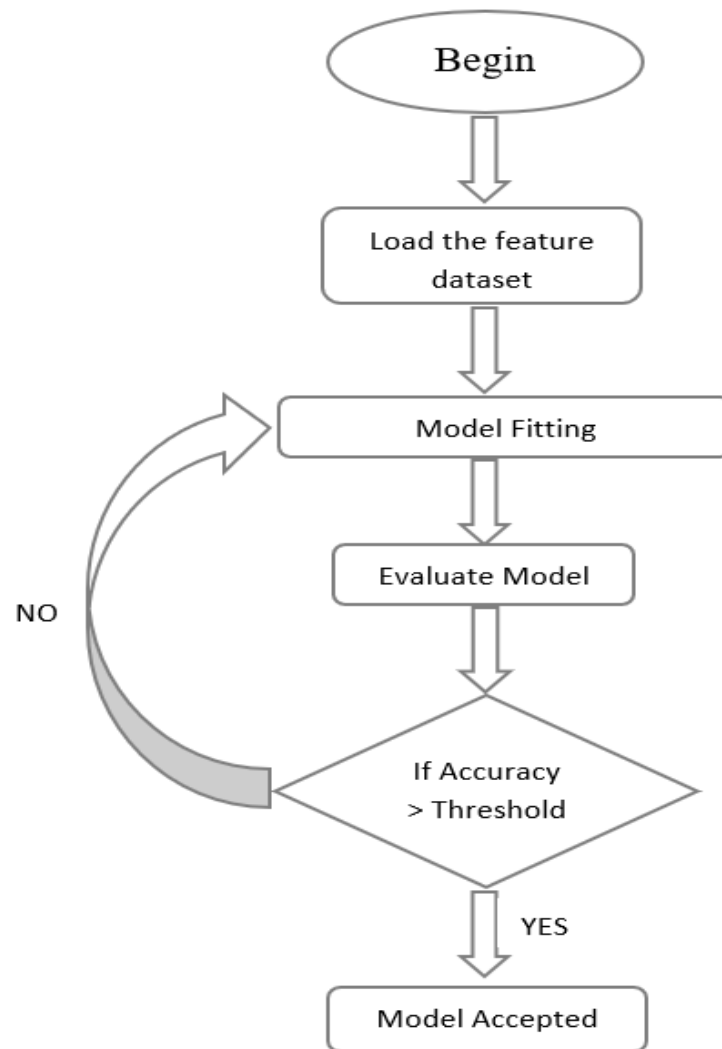
The project was initially developed on a Laptop with Intel(R) Core (TM) i5-8250U CPU @ 1.60GHz, 1801 MHz and was moved to the AWS Sagemaker cloud during the later stages of development. The project has been developed using the programming language python 3.7 with the help of the following libraries and packages as shown in table 5.1.

**Table 5.1: Software Tools**

| Task                             | Library    | Function |
|----------------------------------|------------|----------|
| Plotting graphs and Figures      | Matplotlib | 3.0.1    |
| Natural Language Processing      | NLTK       | 3.3.0    |
| Machine Learning modules         | Scikit     | 0.20.1   |
| Data Array Processing            | NumPy      | 1.16.5   |
| Data structure and analysis tool | Panda      | 0.25.1   |
| Deep learning models             | TensorFlow | 2.4.3    |

## 5.2 Modules

The dataset with ransomware and goodware samples acquired from various sources are uploaded to the Amazon S3 data bucket. The detection system basically consists of three modules namely, the pre-processing, feature engineering and model fitting-evaluation subsystem. The pre-processing module is intended to clean the raw datasets by eliminating duplicates, reducing the dimension of the datasets and by eliminating irrelevant samples. The feature engineering module performs the two major tasks of feature generation and feature selection. The N-gram model and the TF-IDF model is used for the generation of features and the N-gram and TF-IDF scores decide the selection of the features. The model fitting and evaluation module is responsible for training-testing models and identifying a model with the best accuracy. Figure 5.2 captures the flow of the model fitting-evaluation module. The system outputs a model capable of ransomware detection.



**Figure 5.2: Model Evaluation Flowchart**

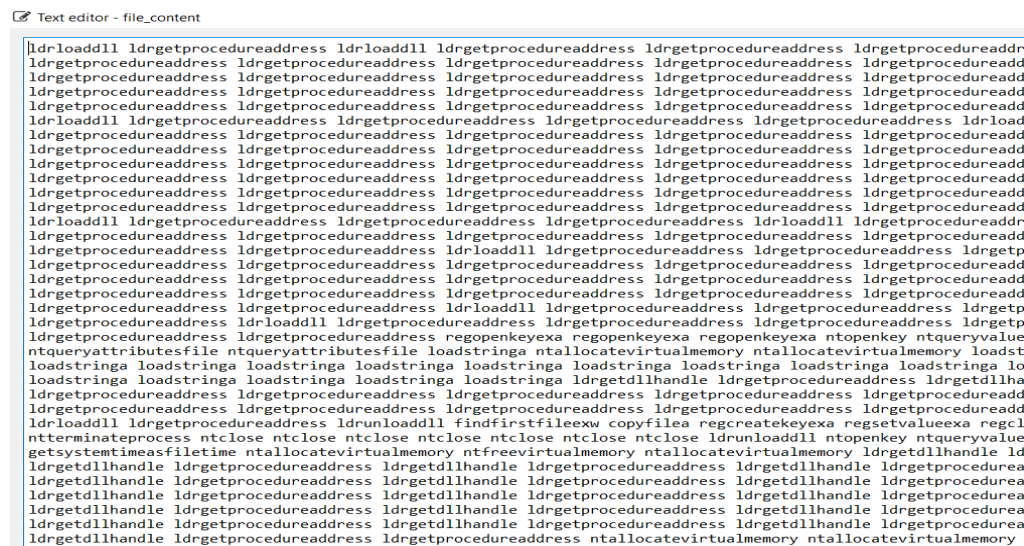
## CHAPTER-6

## RESULTS AND DISCUSSION

This chapter presents the results obtained by the implementation of the system as described in the previous sections. In this section, we discuss the datasets used, the performance metrics and the results of the implementation. The results presented here are further discussed to help understand the importance of the techniques used in implementation of this project.

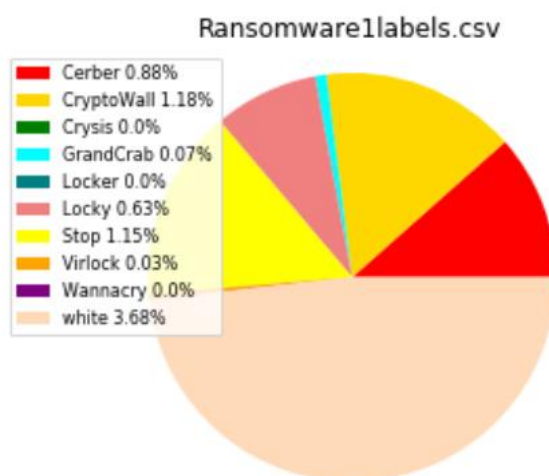
## 6.1 Datasets

The Datasets used in the implementation of this project are obtained two sources majorly, Virus Total and the Open Malware repositories. The dataset obtained are of two types, one dataset consists of the network and static features of the Ransomwares and goodware. The second dataset consists of the Ransomware and goodware API sequences. Figure 6.1 shows a screenshot of the API sequence text file.



**Figure 6.1: API sequence Dataset text file screenshot**

In order to draw insights of the dataset and understand the frequency and availability of different ransomware families, the Class Distribution and the Frequency of the different ransomware families in the dataset is obtained programmatically and is depicted pictorially in the below figures. Figure 6.2 and 6.3 depict the class distribution and frequency distribution of the dataset after the pre-processing step, which helps to understand the dataset better.

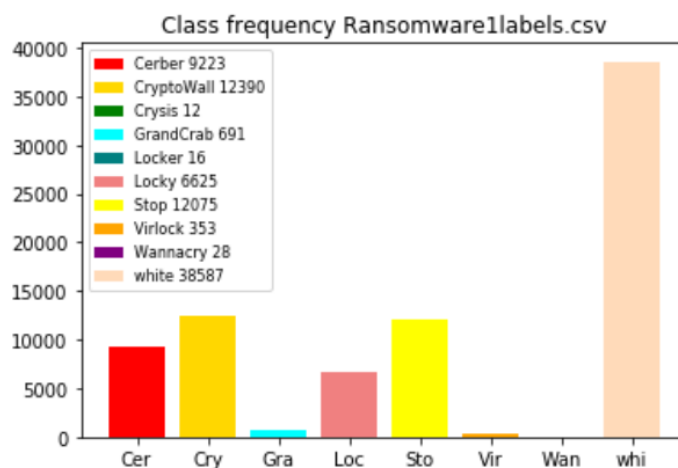


**Figure 6.2: Percentage Distribution of the Dataset**

## 6.2 Pre-Processing Results

The dataset acquired consisted of duplicate records and some samples that were irrelevant to the research. Pre-processing was necessary to enhance the quality of the research and prevent the irrelevant data from misleading the research. Hence the first step of pre-processing involved the elimination of obsolete ransomwares from the dataset and sampling out the ransomware family that are active and massively growing in current times. Keeping in mind the most active and prevalent families of the ransomware, the dataset was sampled. Following which the dimension of the dataset was reduced using the technique of Principle Component analysis (PCA), which rendered a dataset with features or columns that most impacted the outcome of classification. The third and final step in

the pre-processing stage is the filtering of duplicates which helps in efficient use of time and resources. Figure 6.3 depicts the class distribution of the dataset post the pre-processing stage.



**Figure 6.3: Dataset Class Frequency Distribution**

## 6.3 Feature Extraction Results

The Feature extraction results can be broadly classified into the two different approaches used for extraction of features from the dataset, the N-gram model and the TF-IDF model. The results of the same are shared below. The N-gram model is implemented using the API sequence Dataset and the intermediate n-gram dictionary generated is depicted in the Figure 6.4. The screenshot displays a Bigram model where the key represents the Bigrams generated from the API call Dataset and the size attribute specifies the weightage of each of those Bigrams.

igrams - Dictionary (2050 elements)

| Key                                  | Type | Size  |                                                                              |
|--------------------------------------|------|-------|------------------------------------------------------------------------------|
| __anomaly__ ntclose                  | list | 2     | ['ntclose']                                                                  |
| __anomaly__ ntdelayexecution         | list | 29    | ['getcursorpos', 'getcursorpos']                                             |
| __exception__ __anomaly__            | list | 3     | ['ntclose', 'ntdelayexecution', 'ntdelayexecution']                          |
| __exception__ __exception__          | list | 7     | ['ldrloadaddl1', 'ntallocatevirtualmemory', '__exception__', 'ntallocate ... |
| __exception__ coinitalizeex          | list | 1     | ['cocreateinstance', 'cocreateinstance']                                     |
| __exception__ createdirectoryw       | list | 1090  | ['__exception__']                                                            |
| __exception__ createprocessinternalw | list | 43    | ['__exception__', 'ntclose', 'ntclose', '__exception__', '__exception_ ...   |
| __exception__ createthread           | list | 823   | ['ntclose', 'ntclose']                                                       |
| __exception__ drawtextexw            | list | 14991 | ['getsystemmetrics']                                                         |
| __exception__ enumwindows            | list | 821   | ['gethostbyname', 'messageboxtimeouta']                                      |
| __exception__ findfirstfileexw       | list | 40    | ['__exception__', '__exception__', '__exception__', 'ntcreatefile', 'n ...   |
| __exception__ findresourceexw        | list | 159   | ['loadresource']                                                             |
| __exception__ findwindowa            | list | 37    | ['createprocessinternalw', 'createprocessinternalw', 'createprocessint ...   |

**Figure 6.4: N-gram model Implementation result**

The second approach used for feature extraction is the Term Frequency-Inverse Document Frequency (TF-IDF) model, which helps gauge not just the importance of an API in a ransomware sample but also the importance of an API in an entire dataset with both Ransomware and Goodware dataset. The Inverse Document frequency computed for the API in this process is displayed below in the figure 6.5.

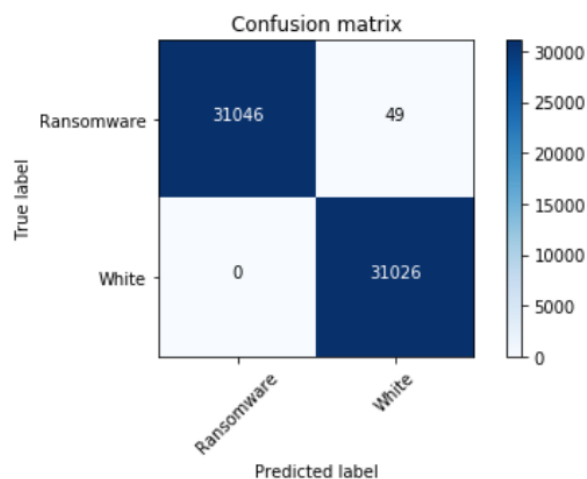
| Key                       | Type | Size | Value                    |
|---------------------------|------|------|--------------------------|
| createdirectoryw          | list | 1    | [0.0001411532218228809]  |
| createprocessinternalw    | list | 1    | [0.00035288305455572025] |
| createreotethread         | list | 1    | [7.057661091114404e-05]  |
| createthread              | list | 1    | [0.0005999011927447244]  |
| createtoolhelp32snapsh... | list | 1    | [7.057661091114404e-05]  |
| cryptacquirecontextw      | list | 1    | [3.528830545557202e-05]  |
| deletefilew               | list | 1    | [3.528830545557202e-05]  |
| deviceiocontrol           | list | 1    | [0.00038817136001129224] |
| drawtextexa               | list | 1    | [0.005681417178347096]   |
| drawtextexw               | list | 1    | [0.011256969440327476]   |
| enumwindows               | list | 1    | [7.057661091114404e-05]  |
| findfirstfileexw          | list | 1    | [0.09856023713741266]    |
| findresourcea             | list | 1    | [0.0003175947491001482]  |

**Figure 6.5: The IDF computed in TF-IDF model**

In the above figure 6.5, the key attribute depicts the API and the Value attribute specifies the corresponding Inverse Document frequency or IDF for the corresponding API.

## 6.4 Random Forest Implementation

The figure 6.6 displays the Confusion matrix for the Ransom Forest model implemented on the Ransomware Dataset with the Bigram features. The Random forest model has performed with an accuracy of 99.92% but the 49 of the Ransomware samples are misclassified as whites which impacts its true positive rate at 99.84%.



**Figure 6.6: Confusion Matrix for Random Forest model**

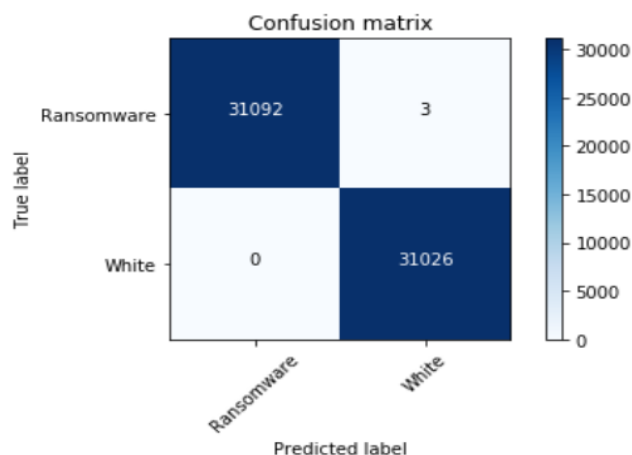
## 6.5 Random Forest with Boosting Implementation

The Random Forest model is boosted with the two major techniques the Adaboost and the Gradient Boost techniques which improves the overall true positive rate for the random forest model with the bigram features. Though the gradient Boost technique did improve the accuracy of the model, the Adaboosting technique helped achieve a perfect accuracy. Figure 6.7 shows the output of the Random Forest with Ada Boosting.

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| Ransomware   | 1.00      | 1.00   | 1.00     | 31095   |
| white        | 1.00      | 1.00   | 1.00     | 31026   |
| micro avg    | 1.00      | 1.00   | 1.00     | 62121   |
| macro avg    | 1.00      | 1.00   | 1.00     | 62121   |
| weighted avg | 1.00      | 1.00   | 1.00     | 62121   |

Confusion matrix, without normalization

```
[[31092  3]
 [  0 31026]]
```



**Figure 6.7: Confusion Matrix for Random Forest model with AdaBoosting**

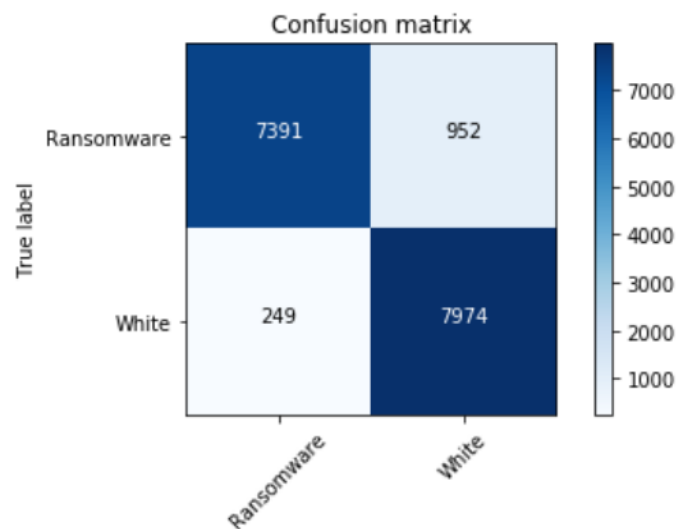
## 6.6 Neural Network Implementation

A simple Multilayer Perceptron with five hidden layers is implemented to classify the ransoms and the goodwares. Figure 6.8 capture the output of the same. Here the Neural Network offers an overall accuracy of 92.75%, considering the rate of false negative weighs down the performance of the Multi-Layer Perceptron.

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| Ransomware   | 0.97      | 0.89   | 0.92     | 8343    |
| white        | 0.89      | 0.97   | 0.93     | 8223    |
| micro avg    | 0.93      | 0.93   | 0.93     | 16566   |
| macro avg    | 0.93      | 0.93   | 0.93     | 16566   |
| weighted avg | 0.93      | 0.93   | 0.93     | 16566   |

Confusion matrix, without normalization

```
[[7391  952]
 [ 249 7974]]
```



**Figure 6.8: Confusion Matrix for Multilayer Perceptron**

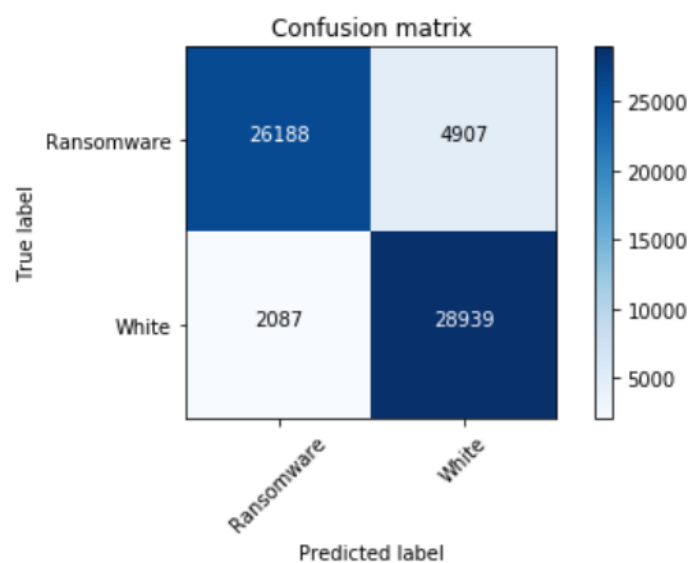
## 6.7 RNN implementation

Figure 6.9 captures the results of the Recurrent Neural Network (RNN) with two dense layers and RELU activation function. The neural network iterates through over two thousand epochs and also uses an Adam optimizer.

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| Ransomware   | 0.93      | 0.84   | 0.88     | 31095   |
| white        | 0.86      | 0.93   | 0.89     | 31026   |
| micro avg    | 0.89      | 0.89   | 0.89     | 62121   |
| macro avg    | 0.89      | 0.89   | 0.89     | 62121   |
| weighted avg | 0.89      | 0.89   | 0.89     | 62121   |

Confusion matrix, without normalization

```
[[26188 4907]
 [ 2087 28939]]
```



**Figure 6.9: Confusion Matrix for Recurrent Neural Network**

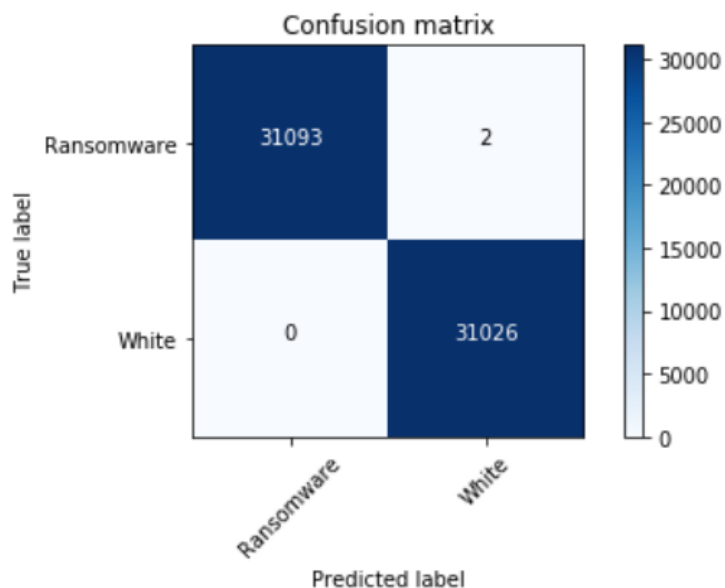
## 6.8 RNN with attention Implementation

Figure 6.10 captures the results of the Recurrent Neural Network (RNN) with two dense layers and a third layer that implements the attention mechanism. The neural network iterates through over two thousand epochs and also uses an Adam optimizer.

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| Ransomware   | 1.00      | 1.00   | 1.00     | 31095   |
| white        | 1.00      | 1.00   | 1.00     | 31026   |
| micro avg    | 1.00      | 1.00   | 1.00     | 62121   |
| macro avg    | 1.00      | 1.00   | 1.00     | 62121   |
| weighted avg | 1.00      | 1.00   | 1.00     | 62121   |

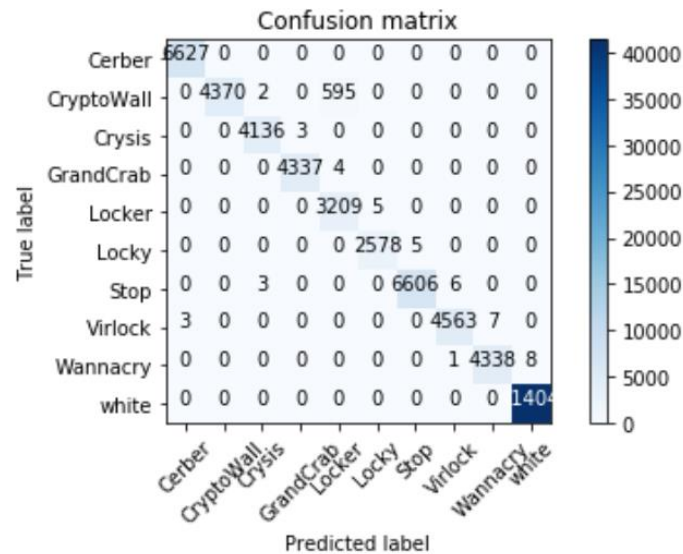
Confusion matrix, without normalization

```
[[31093  2]
 [  0 31026]]
```



**Figure 6.10: Confusion Matrix for Recurrent Neural Network with attention Mechanism**

Apart from the binary classification of the Ransomware and the Goodware samples, the multiclass classification of various Ransomware families holds significance in identifying the different ransomware families. The multiclass classification of various ransomware families using the RNN with attention mechanism is captured by the figure 6.11.



**Figure 6.11: Confusion Matrix for Multiclass classification using Recurrent Neural Network with attention Mechanism**

## 6.9 Discussion

In this section, the results of the various Machine Learning and Deep Learning approaches obtained using the two N-gram and TF-IDF model are compared. The insights gathered throughout the process of execution of the project are discussed and summarized. The importance of the different performance metrics used to assess the Machine Learning and Deep Learning algorithms are also discussed in detail. Table 6.1 tabulates the performance metrics for the Machine learning and Deep Learning models.

Accuracy of a model can be defined as the total number of right predictions made by a model, though this metric forms an important parameter in evaluation of the models, there are other important parameters such as False Positive rate (FPR), Precision, Recall, F1-score, which help in understanding the overall performance of the model. A false positive is a goodware that is falsely identified as a threat. False positive lead to triggering misleading alarm in non-threat situation

**Table 6.1: Table of Classification Metrics**

|                           | Bigram (N=2) |      |      |      |      | Trigram (N=3) |      |      |      |     | TF-IDF |   |     |      |    |
|---------------------------|--------------|------|------|------|------|---------------|------|------|------|-----|--------|---|-----|------|----|
|                           | A            | P    | R    | FPR  | F1   | A             | P    | R    | FPR  | F1  | A      | P | R   | FPR  | F1 |
| <b>RF</b>                 | 98.64        | 1    | 1    | -    | 1    | 99.84         | 1    | 1    | 0.02 | 1   | 97.12  | 1 | 1   | 0.23 | 1  |
| <b>RF with AdaBoost</b>   | 99.01        | 0.8  | 0.7  | -    |      | 99.99         | 1    | 1    | 0.06 | 1   | 98.3   | 1 | 0.8 | 0.7  | 1  |
| <b>MLP</b>                | 92.6         | 0.93 | 0.93 | 0.30 | 0.93 | 91.93         | 0.91 | 0.91 | 0.09 | 0.9 | 91.2   | 1 | 0.7 | 0.03 | 1  |
| <b>RNN</b>                | 86.6         | 0.88 | 0.88 | 0.02 | 0.88 | 88.7          | 0.89 | 0.89 | 0.06 | 0.8 | 86.7   | 1 | 0.8 | 0.02 | 1  |
| <b>RNN with attention</b> | 99.95        | 1    | 1    | 0    | 1    | 99.21         | 1    | 1    | 0    | 1   | 99.01  | 1 | 1   | 0    | 1  |

Table Abbreviations:

A – Accuracy  
P – Precision  
R – Recall  
FPR – False Positive Rate  
F1 – F1 Score  
RF – Random Forest

RF – Random Forest  
MLP – multilevel Perceptron  
RNN – Recurrent Neural Network

leading to an unnecessary incident response. A series of false positive alarms can cause a sense of irritation and in the worst case, a sense of ignorance among the security breach responders for future notifications. Hence False positive rate of a system is an important metric in determining the performance of the system over its lifetime.

Another important factor to consider in the performance of a model is the sample size and the type of sample used in training the model. Comparison of just the detection rates irrespective of the sample size is meaningless. Sample size and samples of ransomware families that are active in the recent years are important considerations for analysis. Apart from the performance of the model, the efficiency of the system in which the models are built and trained are an important factor to consider. The CPU, memory and network resources used by the development system of the Machine learning model and Deep learning models can be enormous due to the size of the training and testing dataset and also the number of iterations the model use to learn. Hence it is necessary to identify efficient

and effective methods for development of the models. The use of Amazon cloud for the development of the models offers an efficient solution to the overload of CPU resources.

The various important points that can be noted from the above results are listed below:

1. The Highest Accuracy is offered by the Random Forest Model that is trained using the Trigram feature set with an accuracy of 99.94% and but also has an FPR of 0.06%.
2. The Second highest accuracy is offered by the RNN with attention mechanism trained with the Bigram features, yet stands out due to its perfect False positive rate.
3. The MLP offers an overall accuracy of 92.6%, but the let-down is the high false positive rate of 0.3.
4. The multiclass Classification of ransomwares offers an accuracy of 96%, while most of the ransomwares could be discriminated by the RNN model, two classes namely, Crypto Wall and Locker were not successfully distinguished due to the similarities in their behaviour during the first few stages in their lifecycle.
5. The use of AWS cloud S3 for storage and the sage maker to train the model, made the process of development faster and the use of RAM efficient.

## CHAPTER-7

### CONCLUSION AND FUTURE WORK

The current Project focuses on training and development of Machine Learning or Deep Learning model that is capable of detecting Ransomwares effectively and efficiently. An important factor in the development of the model was to consider the existing models and implement a model that is at par with the available models but also covers up the existing loopholes of the available models. The intended system is an attempt to extend its model to a better false-positive rate with relevant samples and more efficient and effective development process. The implemented pre-processing and feature engineering steps are expected to contribute to the efficiency of the model. The developed RNN Model with attention Mechanism is capable of detecting of ransomware with a good accuracy and FPR.

In the future, the project can be further extended to use the Honeypots to capture ransomware behaviour and also Network based software for ransomware detection can be merged with this model to further increase their capabilities.

## References

- [1] Ransomware is ‘today’s modern-day extortion,’ McAfee CEO says, by Ryan Browne and Jon Fortt, 2018 <https://www.cnbc.com/2018/02/27/ransomware-is-todays-modern-day-extortion-mcafee-ceo-says.html>.
- [2] 2020 Cyberthreat defense report, CyberEdge Group <https://cyber-edge.com/cdr/#marketing-kit>.
- [3] How Disaster Recovery and Ransomware Intersect: A Disaster Recovery Report December 3, 2019 by Siobhan Climer, Mishaal Khan, and Jason Wankovsky, Mind sight <https://gomindsight.com/insights/blog/disaster-recovery-and-ransomware-ryuk-vcpi/>.
- [4] Malware, ransomware and drive-by download attacks pose biggest cyberthreat challenge in India: Microsoft Security Endpoint Threat Report 2019, Microsoft News centre India, July 29 2020 <https://news.microsoft.com/en-in/microsoft-security-endpoint-threat-report-2019-india/>.
- [5] Norsk Hydro responds to ransomware attack with transparency, Microsoft Security team, <https://news.microsoft.com/transform/hackers-hit-norsk-hydro-ransomware-company-responded-transparency/>.
- [6] Ransomware 2018-2020, Kaspersky Labs [https://media.kasperskycontenthub.com/wp-content/uploads/sites/100/2020/05/12075747/KSN-article\\_Ransomware-in-2018-2020-1.pdf](https://media.kasperskycontenthub.com/wp-content/uploads/sites/100/2020/05/12075747/KSN-article_Ransomware-in-2018-2020-1.pdf).
- [7] Alhawi O.M.K., Baldwin J., Dehghantanha A. (2018) Leveraging Machine Learning Techniques for Windows Ransomware Network Traffic Detection. In: Dehghantanha A., Conti M., Dargahi T. (eds) Cyber Threat Intelligence. Advances in Information Security, vol 70. Springer, Cham.
- [8] Michele Scalas, Davide Maiorca, Francesco Mercaldo, Corrado Aaron Visaggio, Fabio Martinelli, Giorgio Giacinto, On the effectiveness of system API-related information for Android ransomware detection, Computers & Security, Volume 86, 2019, Pages 168-182, ISSN 0167-4048
- [9] J. Stiborek, T. Pevný, and M. Rehák, “Multiple instance learning for malware classification,” Expert Syst. Appl., vol. 93, pp. 346–357, 2018.

- [10] Abdullahi Arabo, Remi Dijoux, Timothee Poulain, Gregoire Chevalier, Detecting Ransomware Using Process Behavior Analysis, *Procedia Computer Science*, Volume 168, 2020, Pages 289-296, ISSN 1877-0509, <https://doi.org/10.1016/j.procs.2020.02.249>.
- [11] S. K. Shaukat and V. J. Ribeiro, "RansomWall: A layered defense system against cryptographic ransomware attacks using machine learning," 2018 10th International Conference on Communication Systems & Networks (COMSNETS), Bengaluru, 2018, pp. 356-363, doi: 10.1109/COMSNETS.2018.8328219.
- [12] S. Egunjobi, S. Parkinson, and A. Crampton, "Classify-ing ransomware using machine learning algorithms," in *Intelligent Data Engineering and Automated Learning– IDEAL 2019*, H. Yin, D. Camacho, P. Tino, A. J. Tallón-Ballesteros, R. Menezes, and R. Allmendinger, Eds. Cham: Springer International Publishing, 2019, pp. 45-52.
- [13] S. Poudyal, K. P. Subedi and D. Dasgupta, "A Framework for Analyzing Ransomware using Machine Learning," 2018 IEEE Symposium Series on Computational Intelligence (SSCI), Bangalore, India, 2018, pp. 1692-1699, doi: 10.1109/SSCI.2018.8628743.
- [14] K. Lee, S. Lee and K. Yim, "Machine Learning Based File Entropy Analysis for Ransomware Detection in Backup Systems," in *IEEE Access*, vol. 7, pp. 110205-110215, 2019, doi: 10.1109/ACCESS.2019.2931136.
- [15] F. Khan, C. Ncube, R. L. Kumar, S. Kadry and Y. Nam, "A Digital DNA Sequencing Engine for Ransomware Detection Using Machine Learning," in *IEEE Access*, doi: 10.1109/ACCESS.2020.3003785.
- [16] Greg Cusack, Oliver Michel, and Eric Keller. 2018. Machine Learning-Based Detection of Ransomware Using SDN. In *Proceedings of the 2018 ACM International Workshop on Security in Software Defined Networks & Network Function Virtualization (SDN-NFV Sec'18)*. Association for Computing Machinery, New York, NY, USA, 1–6. DOI: <https://doi.org/10.1145/3180465.3180467>.
- [17] R. Vinayakumar, K. P. Soman, K. K. Senthil Velan and S. Ganorkar, "Evaluating shallow and deep networks for ransomware detection and classification," 2017 International Conference on Advances in Computing, Communications and Informatics (ICACCI), Udupi, 2017, pp. 259-265, doi: 10.1109/ICACCI.2017.8125850.
- [18] Bin Zhang, Wentao Xiao, Xi Xiao, Arun Kumar Sangaiah, Weizhe Zhang, Jiajia Zhang, Ransomware classification using patch-based CNN and self-attention network on embedded N-grams of opcodes, *Future Generation Computer Systems*, Volume 110, 2020, Pages 708-720, ISSN 0167-739X, <https://doi.org/10.1016/j.future.2019.09.025> <http://www.sciencedirect.com/science/article/pii/S0167739X19315912>.

- [19] M. Rhode, P. Burnap, and K. Jones, "Early-stage malware prediction using recurrent neural networks," *Comput. Secur.*, vol. 77, no. December 2017, pp. 578–594, 2018.
- [20] Sajad Hodayoun, Ali Dehghantanha, Marzieh Ahmadzadeh, Sattar Hashemi, Raouf Khayami, Kim-Kwang Raymond Choo, David Ellis Newton, DRTHIS: Deep ransomware threat hunting and intelligence system at the fog layer, *Future Generation Computer Systems*, Volume 90, 2019, Pages 94-104, ISSN 0167-739X, <https://doi.org/10.1016/j.future.2018.07.045>.  
<http://www.sciencedirect.com/science/article/pii/S0167739X17328467>.
- [21] R. Agrawal, J. W. Stokes, K. Selvaraj and M. Marinescu, ""Attention in Recurrent Neural Networks for Ransomware Detection,"" ICASSP 2019 - 2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), Brighton, United Kingdom, 2019, pp. 3222-3226, doi: 10.1109/ICASSP.2019.8682899."
- [22] H. Jinsoo, K. Jeankyung, S. Lee, and K. Kim, "Two-stage ransomware detection using dynamic analysis and machine learning techniques," 2020, pp. 2597–2609 [Online]. Available: <https://doi.org/10.1007/s11277-020-07166-9>.
- [23] M. Al-Hawawreh and E. Sitnikova, ""Leveraging Deep Learning Models for Ransomware Detection in the Industrial Internet of Things Environment,"" 2019 Military Communications and Information Systems Conference (MilCIS), Canberra, Australia, 2019, pp. 1-6, doi: 10.1109/MilCIS.2019.8930732."
- [24] Hanqi Zhang, Xi Xiao, Francesco Mercaldo, Shiguang Ni, Fabio Martinelli, Arun Kumar Sangaiah, Classification of ransomware families with machine learning based on N-gram of opcodes, *Future Generation Computer Systems*, Volume 90, 2019, Pages 211-221, ISSN 0167-739X <https://doi.org/10.1016/j.future.2018.07.052>.
- [25] Tarfa Hamed, Rozita Dara, Stefan C. Kremer, Network intrusion detection system based on recursive feature addition and bigram technique, *Computers & Security*, Volume 73, 2018, Pages 137-155, ISSN 0167-4048, <https://doi.org/10.1016/j.cose.2017.10.011>.  
<http://www.sciencedirect.com/science/article/pii/S016740481730227>.
- [26] S. Poudyal, D. Dasgupta, Z. Akhtar and K. Gupta, ""A multi-level ransomware detection framework using natural language processing and machine learning"", 14th International Conference on Malicious and Unwanted Software", 2019.
- [27] Chen Q., Islam S.R., Haswell H., Bridges R.A. (2019) Automated Ransomware Behavior Analysis: Pattern Extraction and Early Detection. In: Liu F., Xu J., Xu S., Yung M. (eds) *Science of Cyber Security. SciSec 2019. Lecture Notes in Computer Science*, vol 11933. Springer, Cham.

- [28] L. Y. Connolly and D. S. Wall, “The rise of crypto-ransomware in a changing cybercrime landscape: Taxonomizing countermeasures,” *Computers & Security*, vol. 87, p. 101568, 2019.
- [29] E. Kalaimannan, S. K. John, T. DuBose, and A. Pinto, “Influences on ransomware’s evolution and predictions for the future challenges,” *Journal of Cyber Security Technology*, vol. 1, no. 1, pp. 23–31, 2017.
- [30] D. Kao and S. Hsiao, “The dynamic analysis of wannacry ransomware,” in 2018 20th International Conference on Advanced Communication Technology (ICACT), 2018, pp. 159–166
- [31] A. Adamov and A. Carlsson, “The state of ransomware. Trends and mitigation techniques,” in 2017 IEEE East-West Design Test Symposium (EWDTS), 2017, pp. 1–8.
- [32] H. Kok, A. Abdullah, Z. Jhanjhi, and M. Supramaniam, “Ransomware, threat and detection techniques: A review.” *Int. J. Comput. Sci. Netw. Secur*, 2019.
- [33] E. Berrueta, D. Morato, E. Magaña, and M. Izal, “A survey on detection techniques for cryptographic ransomware,” *IEEE Access*, vol. 7, pp. 144 925–144 944, 2019.